

程序：psi_dynamics.py 说明

1. 程序功能概述

psi_dynamics.py 是 KnotSim 框架的动力学模块，它实现了编织强度 Ψ 的时间演化，直接对应离散关联网络公理体系中的动力学方程：

$$i\hbar \frac{\partial \Psi}{\partial t} = \alpha L\Psi - \beta |\Psi|^2 + \gamma C$$

程序基于欧拉数值积分，对编织强度矩阵进行单步演化，模拟拓扑耦合的动态变化，为后续拓扑结识别与稳定性分析提供演化后的网络状态。

2. 代码与含义解释

2.1 依赖与基础函数

python

运行

```
import numpy as np
```

```
import random
```

```
def generate_discrete_network(num_nodes, connection_prob=0.3):
```

```
    # 与 network_generator.py 中的实现完全一致
```

```
    nodes = list(range(num_nodes))
```

```
    adjacency_matrix = np.zeros((num_nodes, num_nodes), dtype=complex)
```

```
    edges = []
```

```
    for i in range(num_nodes):
```

```
        for j in range(i + 1, num_nodes):
```

```
            if random.random() < connection_prob:
```

```
                psi = np.exp(1j * random.uniform(0, 2 * np.pi))
```

```
                adjacency_matrix[i, j] = psi
```

```
                adjacency_matrix[j, i] = np.conj(psi)
```

```
                edges.append((i, j))
```

```
return nodes, edges, adjacency_matrix
```

- 导入 numpy 用于复数矩阵运算，random 用于生成初始网络。
- generate_discrete_network: 与前序模块复用，生成初始复数邻接矩阵（编织强度矩阵）。

2.2 动力学演化函数

python

运行

```
def psi_dynamics_step(adjacency_matrix, hbar=1.0, alpha=0.1, beta=0.01,
gamma=0.001, dt=0.01):

    N = adjacency_matrix.shape[0]

    # 1. 构造度矩阵 D 和拉普拉斯矩阵 L

    D = np.diag(np.sum(np.abs(adjacency_matrix), axis=1))

    L = D - adjacency_matrix

    # 2. 拓扑荷项 C（相位缠绕数近似）

    phase = np.angle(adjacency_matrix)

    C = np.sum(phase, axis=1, keepdims=True) @ np.ones((1, N)) / (2 * np.pi)

    # 3. 计算动力学方程右侧

    dPsi_dt = (1 / (1j * hbar)) * (

        alpha * (L @ adjacency_matrix)          # 扩散项：拉普拉斯算子驱动

        - beta * (np.abs(adjacency_matrix)**2) * adjacency_matrix  # 非线性自洽项

        + gamma * C                               # 拓扑荷项：相位缠绕的拓扑驱动

    )

    # 4. 欧拉单步演化

    adjacency_matrix_new = adjacency_matrix + dPsi_dt * dt

    return adjacency_matrix_new
```

- 输入参数：
 - adjacency_matrix: 初始编织强度复数矩阵；

- $\hbar$: 普朗克常数 (无量纲化, 默认 1.0);
- α : 扩散项系数, 控制拉普拉斯驱动的耦合强度;
- β : 非线性项系数, 控制自洽耦合的强度;
- γ : 拓扑荷项系数, 控制相位缠绕对耦合的影响;
- Δt : 时间步长, 控制数值演化精度。

- **关键步骤解释:**

1. **拉普拉斯矩阵 $L=D-A$:**

- D 为度矩阵, 对角元为节点耦合强度之和;
- A 为编织强度邻接矩阵;
- L 作为扩散算子, 驱动耦合强度在网络上扩散与重分布。

2. **拓扑荷项 C :**

- 提取邻接矩阵的相位, 计算相位缠绕数近似值;
- 该项对应拓扑耦合的固有自由度, 驱动编织强度向拓扑稳定态演化。

3. **欧拉演化:**

- 对动力学方程进行一阶欧拉步进, 得到演化后的编织强度矩阵;
- 数值格式为: $\Psi(t + \Delta t) = \Psi(t) + \frac{d\Psi}{dt} \cdot \Delta t$ 。

2.3 主函数测试

python

运行

```
if __name__ == "__main__":
    nodes, edges, adj = generate_discrete_network(10, 0.3)

    print("节点数:", len(nodes))

    print("关联数:", len(edges))

    print("初始邻接矩阵:\n", adj)
```

```
adj_evolved = psi_dynamics_step(adj)

print("\n 演化后邻接矩阵:\n", adj_evolved)
```

- 生成 10 节点、关联概率 0.3 的初始网络;
 - 执行一次动力学演化;
 - 打印初始与演化后的编织强度矩阵, 对比耦合强度与相位的变化。
-

3. 运行输出结果解释

截图中的输出示例:

plaintext

节点数: 10

关联数: 12

邻接矩阵:

```
[[ 0.+0.j  0.9858569-0.16758931j  0.+0.j ... ]]
```

- 节点数 / 关联数: 与初始网络生成模块一致, 描述网络规模与连通性;
 - 初始邻接矩阵: 复数矩阵, 元素为编织强度 Ψ , 模长为耦合强度, 相位为拓扑方向;
 - 演化后邻接矩阵: 矩阵元素的模长与相位发生变化, 体现了扩散、非线性自洽与拓扑荷项的综合驱动效果。
-

4. 与理论框架的关联

- 本程序实现了离散关联网络公理体系中的动力学演化方程, 将抽象的耦合演化过程转化为可计算的数值模拟;
- 演化后的编织强度矩阵, 是后续 knot_detector.py 中拓扑结识别与稳定性判定的核心输入;
- 通过调整 $\alpha/\beta/\gamma$ 系数, 可模拟不同拓扑耦合场景下的演化行为, 验证理论预言的拓扑稳定态。